

ROTEIRO DO CURSO

1

Introdução ao Python

2

Operações de Entrada/Saída

3

Estruturas Condicionais e de Repetição

4

Funções e Estruturas de Dados

OBJETIVOS DA AULA

1. Conhecer as funções.
2. Entender sobre o escopo de variáveis.
3. Entender o que são as exceções.
4. Realizar o tratamento de exceções.

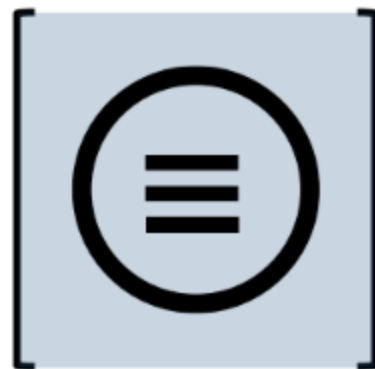


CONHECER AS FUNÇÕES

CONHECER AS FUNÇÕES

Funções são blocos de código reutilizáveis que executam uma tarefa específica quando são chamados.

Ajudam a dividir o código em partes menores e mais gerenciáveis, tornando o software mais organizado e fácil de manter.



CONHECER AS FUNÇÕES

De onde vem as funções do Python?
Basicamente, podem ter 3 origens:

EMBUTIDAS

Pode ser incorporada em um código e são nativas da linguagem de programação. Não é preciso importá-la.

MÓDULOS

São incorporadas nos códigos a partir da importação ou da referência de módulos externos do Python.

AUTORIA PRÓPRIA

São escritas pelo desenvolvedor do software. Servem para organizar e reaproveitar melhor o código.

CONHECER AS FUNÇÕES

FUNÇÃO SEM RETORNO DECLARAÇÃO

- Use a palavra reservada **DEF**.
- Dê um nome.
- Defina os parâmetros.
- Coloque os dois pontos.
- Escreva o código.

```
1  # Procedimento que apresenta uma mensagem.
2  def apresentarMensagem (mensagem):
3      |    print(mensagem)
4
5  # Chamada para o procedimento.
6  apresentarMensagem("Olá, mundo!!!")
```

CONHECER AS FUNÇÕES

FUNÇÃO COM RETORNO DECLARAÇÃO

- Use a palavra reservada **DEF**.
- Dê um nome.
- Defina os parâmetros.
- Coloque os dois pontos.
- Escreva o código.
- Defina o retorno (**RETURN**).

```
1  # Função que apresenta uma soma.
2  def somar (numero1, numero2):
3      soma = numero1 + numero2
4      return soma
5
6  # Chamada para a função.
7  somar(15 , 10)
8  print(somar)
```

CONHECER AS FUNÇÕES

FUNÇÕES EM GERAL PARÂMETROS

Além de executar códigos, as funções também podem receber e retornar dados. Os parâmetros permitem interagir de forma personalizada com as funções.

```
1  # Procedimento que apresenta uma mensagem.
2  # O nome do usuário é um parâmetro.
3  def apresentarMensagem (nome):
4      |    print(f"Seja bem vindo: {nome} !!!")
5
6  # Chamada para o procedimento.
7  # Passagem do nome como parâmetro.
8  apresentarMensagem("Marcus Vinícius")
```

CONHECER AS FUNÇÕES

FUNÇÕES EM GERAL VALOR PADRÃO

Pode-se definir um valor padrão para os parâmetros de uma função, para prevenir erros em tempo de execução do código (Exceções).

```
1  # Função que define a cor de uma flor.
2  # O parâmetro é a cor.
3  def colorirFlor (cor = 'Preta'):
4      print(f"A cor da flor é {cor}")
5
6  # Chamada para a função.
7  # Passagem da cor como parâmetro.
8  colorirFlor()
9  colorirFlor('Vermelha')
```

CONHECER AS FUNÇÕES

FUNÇÕES EM GERAL CHAMADA POSICIONAL

A chamada posicional de uma função é quando se coloca os parâmetros na mesma ordem (**POSIÇÃO**) em que eles foram declarados.

```
1  # Função que a área de um retângulo.
2  def calcularArea (base, altura):
3      area = base * altura
4      return area
5
6  # Chamada para a função.
7  calcularArea(8, 5)
8  print(calcularArea)
```

CONHECER AS FUNÇÕES

FUNÇÕES EM GERAL CHAMADA NOMEADA

A chamada nomeada de uma função é quando se coloca o nome e depois o valor dos parâmetros. Não importa a ordem do parâmetro.

```
1  # Função que a área de um retângulo.
2  def calcularArea (base, altura):
3      area = base * altura
4      return area
5
6  # Chamada para a função.
7  calcularArea(base=8, altura=5)
8  print(calcularArea)
```



ENTENDER SOBRE O ESCOPO DE VARIÁVEIS

ENTENDER SOBRE O ESCOPO DE VARIÁVEIS

ESCOPO DE VARIÁVEIS LOCAL

- Variáveis declaradas dentro da função têm escopo local.
- Só podem ser acessadas e usadas dentro da função.
- No término de execução, deixam de existir.

```
1  # Função que calcula um desconto.
2  def calcularDesconto (valor):
3      taxa = 100.00
4      final = valor - taxa
5      return final
6
7  # Chamada da função.
8  Valor = calcularDesconto(250.00)
9  print(calcularDesconto)
```

ENTENDER SOBRE O ESCOPO DE VARIÁVEIS

ESCOPO DE VARIÁVEIS GLOBAL

- Variáveis declaradas fora de funções tem escopo global.
- Podem ser acessadas e usadas em qualquer lugar.
- Usando o termo **GLOBAL** é possível modificá-la.

```
1  # Variável global.
2  contador = 0
3
4  # Função de incremento.
5  def incrementar():
6      global contador
7      contador += 1
8
9  # Chamada da função.
10 incrementar()
11 print(contador)
```



**ENTENDER O QUE
SÃO AS EXCEÇÕES**

ENTENDER O QUE SÃO AS EXCEÇÕES

Existem diferentes tipos de erros na programação de softwares:

SINTAXE

É quando se erra a gramática da linguagem de programação, ou seja, se escreve um comando que não existe. É como um erro de digitação.

LÓGICA

Conhecidos como bugs, é quando a sintaxe executa sem erros, mas o código não está produzindo o resultado esperado.

RUNTIME

Ocorrem durante a execução do código. Estão relacionados a uma operação incorreta por meio do usuário. São conhecidos como **exceções**.

ENTENDER O QUE SÃO AS EXCEÇÕES

Neste sentido, é necessário entender as exceções que existem para tratá-las:

KEY ERROR

Ocorre quando uma chave não existe em um dicionário.

INDEX ERROR

Ocorre quando é feita uma indexação fora do intervalo válido em uma sequência, como em listas, tuplas e outras estruturas de dados.

FILE NOT FOUND ERROR

Ocorre quando um arquivo não pode ser encontrado.

ENTENDER O QUE SÃO AS EXCEÇÕES

Neste sentido, é necessário entender as exceções que existem para tratá-las:

TYPE ERROR

Ocorre quando uma função recebe um argumento com um valor inapropriado.

VALUE ERROR

Ocorre quando uma operação é realizada em um tipo de dado incompatível.

ZERO DIVISION ERROR

Ocorre quando uma divisão ou uma operação matemática de módulo é realizada por zero.

ENTENDER O QUE SÃO AS EXCEÇÕES

Neste sentido, é necessário entender as exceções que existem para tratá-las:

NAME ERROR

Ocorre quando o Python não encontra uma variável ou algum nome definido no código.

EXCEPTION

É um termo usado como um "coringa" para capturar qualquer tipo de exceção que possa ocorrer. Ou seja, pode ser usado para capturar exceções não esperadas.

IMPORT ERROR

Ocorre quando há um erro ao tentar realizar a importação de algum módulo/pacote externo do Python.



REALIZAR O TRATAMENTO DE EXCEÇÕES

REALIZAR O TRATAMENTO DE EXCEÇÕES

TRY-EXCEPT SINTAXE

O bloco de código básico que realiza o tratamento de exceções no Python é o **TRY-EXCEPT**.

```
1  # Faz a tentativa de execução de um código.  
2  try:  
3      print("Código a ser executado...")  
4  
5  # Caso ele não execute, captura a exceção.  
6  except Exception:  
7      print("Executa aqui em caso de exceção...")
```

REALIZAR O TRATAMENTO DE EXCEÇÕES

TRY-EXCEPT

CAPTURA DE EXCEÇÃO

Capturar uma exceção é pegar o erro que foi lançado pelo código e tratá-lo de forma adequada, evitando que ela interrompa abruptamente a execução do software.

```
1  # Tentativa de execução do código.
2  try:
3      dividendo = int(input("Digite: "))
4      divisor = int(input("Digite: "))
5      resultado = dividendo/divisor
6
7  # Trata uma exceção, caso ela exista.
8  except ZeroDivisionError:
9      print("Não há divisão por zero!!!")
10
```

REALIZAR O TRATAMENTO DE EXCEÇÕES

TRY-EXCEPT

TRATAMENTO MÚLTIPLO

Também é possível capturar múltiplas exceções, evitando assim que o software tenha sua execução interrompida devido a diferentes tipos de falhas.

```
1  # Tentativa de execução do código.
2  try:
3      dividendo = int(input("Digite: "))
4      divisor = int(input("Digite: "))
5      resultado = dividendo/divisor
6
7  # Tratamento de duas exceções.
8  except ValueError:
9      print("Número inválido!!!")
10
11 except ZeroDivisionError:
12     print("Não há divisão por zero!!!")
```